

E-morphic: Scalable Equality Saturation for Structural Exploration in Logic Synthesis

Chen Chen*

HKUST(GZ)

cchen099@connect.hkust-gz.edu.cn

Guangyu Hu*[†]

HKUST

ghuae@connect.ust.hk

Cunxi Yu

University of Maryland, College Park

cunxiyu@umd.edu

Yuzhe Ma

HKUST(GZ)

yuzhema@hkust-gz.edu.cn

Hongce Zhang[†]

HKUST(GZ)

hongcezh@hkust-gz.edu.cn

Abstract—In technology mapping, the quality of the final implementation heavily relies on the circuit structure after technology-independent optimization. Recent studies have introduced equality saturation as a novel optimization approach. However, its efficiency remains a hurdle against its wide adoption in logic synthesis. This paper proposes a highly scalable and efficient framework named **E-morphic**. It is the first work that employs equality saturation for resynthesis after conventional technology-independent logic optimizations, enabling structure exploration before technology mapping. Powered by several key enhancements to the equality saturation framework, such as direct e-graph-circuit conversion, solution-space pruning, and simulated annealing for e-graph extraction, this approach not only improves the scalability and extraction efficiency of e-graph rewriting but also addresses the structural bias issue present in conventional logic synthesis flows through parallel structural exploration and resynthesis. Experiments show that, compared to the state-of-the-art delay optimization flow in ABC, **E-morphic** on average achieves 12.54% area saving and 7.29% delay reduction on the large-scale circuits in the EPFL benchmark.

I. INTRODUCTION

Typically, technology mapping employs graph or tree covering algorithms to map an And-Inverter Graph (AIG) into standard cells. However, technology mapping based on structural covering faces a challenge known as structural bias [1]. Namely, the quality of result (QoR) is largely dependent on the structure of subject graphs [2]. If the initial structure is poor, the final mapping will also be sub-optimal, even with the use of heuristics or iterative recovery methods.

To address the structural bias problem, previous research introduced lossless synthesis [3] to explore a few structural choices during mapping. It employs heuristic rewriting to produce functional equivalent nodes that are then combined into a single subject graph with choices, which is subsequently used to derive the mapped netlist and researches [1] [4] [5] targeted at mitigating structural bias heuristically, they cannot avoid this issue completely.

Recently, equality saturation has emerged as a promising technique, which uses a non-destructive rewriting approach to achieve better Pareto-optimal design space exploration and has

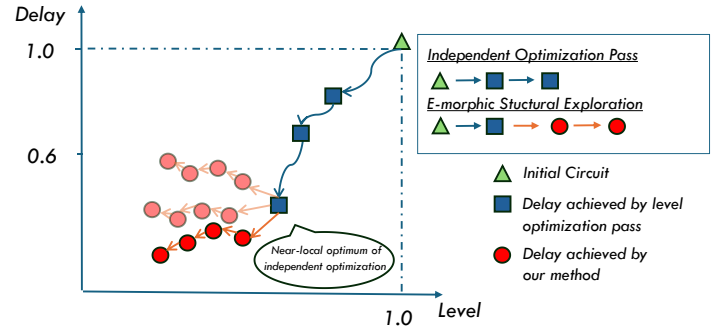


Fig. 1: E-morphic: parallel structural exploration for delay finetuning improves the circuit performance.

been widely applied in the EDA field [6]. The applications include program optimization [7], compiling optimization [8], [9], datapath optimization [10], multiplier optimization [11], etc. Among these prior works, E-Syn [12] first applied e-graph rewriting to Boolean logic optimization and demonstrated the potential of e-graphs in exploring optimal logic structure. However, due to the complexity of e-graph-based optimization, these previous works have to be on a limited scale, with no more than 40,000 e-nodes [10]. When the graph scale significantly grows, the two main steps in equality saturation, which are rewriting (to expand solution space) and extraction (to pick a good solution), would inevitably suffer from excessively long runtime. Therefore, it is still an unanswered question whether e-graphs can play an effective role in large-scale Boolean logic optimization scenarios.

As we work to scale up equality saturation for logic synthesis, we acquire the following two key insights. (1) The method [1] [3] faces a primary challenge: because ABC inherently performs local rewriting [13], previous approaches do not maintain equivalence classes during the rewriting process. Instead, they create equivalence classes by detecting equivalent nodes through simulation and SAT checking across multiple circuits after performing local rewrites against the original circuit. This approach maintains only one additional equivalent node per equivalence class in the most commonly used ABC operator `dch` and has a restricted ability to make global structural changes. However, e-graphs utilize congruence closure-based [14] non-destructive rewriting, fundamentally aiming to maintain a vast

This work is supported in part by the National Natural Science Foundation of China under Grant No. 62304194, and by Guangzhou Municipal Science and Technology Project under Grant No. 2023A03J0013.

* Both authors contributed equally to this research.

[†]Corresponding author.

number of equivalence classes. **By performing only a small number of iterations of e-graph-based rewriting, e-graphs can generate significantly more equivalence classes than previous methods, enabling larger structural transformations. Additionally, using fewer iterations ensures that independent optimization targets such as node sizes and logic levels are not adversely affected.** Through a case study presented in Figure 1, we observed that as technology-independent optimizations approach a near-local optimum, our tool, E-morphic, employs parallel e-graph-based resynthesis to mitigate structural bias issues. This approach facilitates the exploration of more optimal solutions for technology mapping targets. (2) Although fewer iterations already generate an excessive number of equivalence classes and nodes, the presence of too many equivalent nodes within each class severely impacts the time required for e-graph extraction. **Therefore, it is important to have an efficient and effective e-graph extraction method to explore various structural choices within a reasonable time. To this end, we incorporate a series of new features into the equality saturation framework, including solution space pruning and simulated annealing. Solution space pruning effectively reduces the over-abundance of redundant equivalent nodes in each equivalence class, significantly decreasing the time required for e-graph extraction while simulated annealing enables the escape from local minima when selecting from equivalent terms.**

Bearing these ideas in mind, we build an equality saturation framework named E-morphic for practical Boolean logic synthesis problems. It uses equality saturation in a fast and scalable manner to address the efficiency and scalability issues. The word “morphic” in its name suggests its intended role in logic synthesis — exploring diverse logic structures to improve the post-mapping design quality.

The contributions of this paper are as follows:

- To our best knowledge, this is the first work that adopts e-graph rewriting to address the structural bias issue of technology mapping in logic synthesis.
- It brings up several important updates to the equality saturation framework, such as DAG-to-DAG conversion, solution space pruning, simulated annealing, and multi-batch parallel computing for e-graph extraction. These techniques notable enhance the efficiency of equality saturation in logic synthesis and could be helpful for other e-graph applications.
- We implement a scalable and effective logic optimization framework and achieve 12.54% area saving and 7.29% delay reduction compared to a competitive delay-optimal synthesis flow on the large-scale circuits in the EPFL benchmark.

II. BACKGROUND

A. E-Graph and Equality Saturation

An e(quality)-graph is a data structure used to represent a set of equivalent terms (expressions). Figure 2 gives an example of an e-graph where each green node is called an e-node that represents a term from a given language. The red-dotted boxes are equivalent classes (e-classes) where e-nodes in the same

TABLE I: Examples of rewriting rules

Class	Pattern (LHS)	Transformation (RHS)
Commutativity	$a * b$	$b * a$
	$a + b$	$b + a$
Associativity	$(a * b) * c$	$a * (b * c)$
	$(a + b) + c$	$a + (b + c)$
Distributivity	$a * (b + c)$	$a * b + a * c$
	$(a + b) * (a + c)$	$a + (b * c)$
	$(a * b) + (a * c)$	$a * (b + c)$
Consensus	$(a * b) + ((-a) * c) + b * c$	$(a * b) + (-a) * c$
	$((a + b) * ((-a) + c)) * (b + c)$	$(a + b) * ((-a) + c)$
De-Morgan	$\neg(a * b)$	$\neg a + \neg b$
	$\neg(a + b)$	$(-a) * (-b)$

class represent equivalent expressions. E-graph also maintains congruence relations¹ over e-classes.

For a given input term, we may rewrite it into other equivalent forms following a set of rules (such as those listed in Table I). This process of finding equivalent terms is called equality saturation. Given a term $\mathcal{T}$, an initial e-graph $\mathcal{E}$ is constructed. Then we can iteratively find patterns that match with the left-hand-side of rewriting rules and record the right-hand-side patterns that will be added to the graph. Specifically, the rewriting does not remove the original term but only adds information to $\mathcal{E}$. Therefore, it is non-destructive and the ordering of rewriting will not affect the outcome. This solves the so-called phase ordering problem in optimization. The rewriting iterations may terminate when no more rule is applicable (namely, the graph is saturated) or when a certain exiting condition is met. Then an optimized term can be extracted from the final e-graph where the optimality of terms is measured by a cost function defined w.r.t. the whole graph.

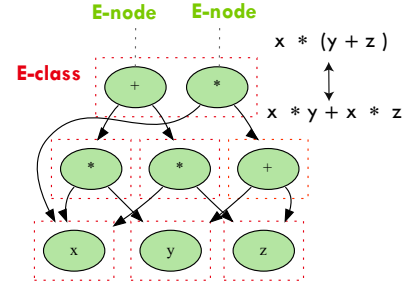


Fig. 2: This e-graph represents two equivalent expressions. Each green node is an e-node, and each red-dotted box is an e-class. Edges connect e-nodes to child e-classes.

B. E-Graph Rewriting in Logic Synthesis

E-graph has demonstrated its potential in many optimization problems [8]–[11], [15], while the prior work E-Syn [12] first applied equality saturation in logic synthesis. E-Syn is based on an existing e-graph processing framework egg [16], as presented by Figure 3. Its workflow is as follows: **1 Pre-processing:** E-Syn takes advantage of the ABC logic synthesis tool to transform input Verilog or AIG into the equation format. Then E-Syn converts the equation format into the S-expression, which is a data structure for nested lists commonly used in Lisp-like programming languages and can be converted into e-graphs using the built-in functionality of egg. **2 Rewriting:** The egg

¹If two terms a and b are equivalent, any function applications over a and b would also be equivalent ($f(a) \equiv f(b)$).

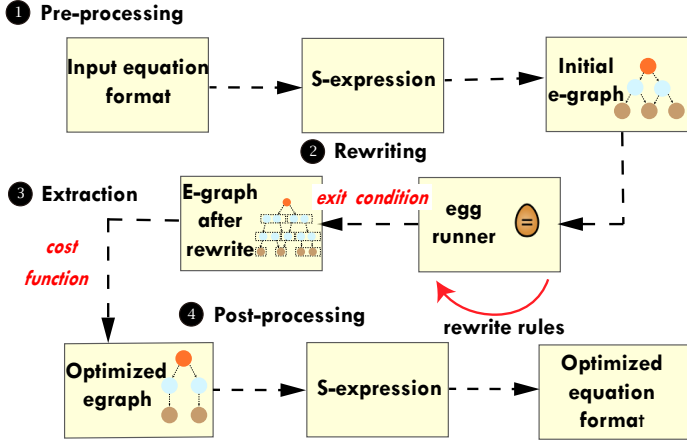


Fig. 3: The workflow of E-Syn [12]

runner will iteratively apply a set of rewriting rules until the e-graph is saturated or an exit condition is met, such as when the time limit or maximum iteration is reached. **3 Extraction:** It selects the best term based on a cost function. To achieve overall algorithmic efficiency, a commonly used approach is to traverse the e-graph bottom-up and greedily select the e-node with the lowest cost among the equivalent ones in the same e-class. **4 Post-processing:** E-Syn finally converts the optimized e-graph back into the equation format, which can be mapped to gate-level circuits using synthesis tools. Then the performance metrics of the circuit will be reported.

III. METHODOLOGY

E-morphic effectively addresses several significant challenges associated with previous works [12], [17], which struggles with problems like inefficient intermediate representations and restricted cost functions, leading to scalability issues. Our solution overcomes these limitations through critical advancements, which we will discuss in the following subsections.

A. E-morphic Overview

Our framework, E-morphic, introduces a novel and efficient method for applying e-graphs in logic synthesis. We identify a unique role for e-graph-based optimization, between technology-independent optimization and technology mapping. E-morphic offers several key advantages:

- 1) **Structural Exploration before Technology Mapping:** Parallel E-graph-based resynthesis using fewer rewriting iterations after technology-independent optimization enables efficient structural exploration to address the structural bias issue.
- 2) **Novel Extraction Method:** Integrating Solution Space Pruning and Simulated Annealing-Based Algorithms with Multi-Threaded Parallel Computing for Efficient E-graph Extraction and Faster Evaluation.
- 3) **Efficient Implementation:** Our framework incorporates several efficiency-enhancing features: (a) direct DAG-to-DAG conversion between circuits and e-graphs, eliminating the overhead of intermediate S-expressions, and (b) machine learning-based cost estimation for rapid quality assessment.

Figure 5 illustrates the workflow of E-morphic. The input circuit first undergoes conventional technology-independent optimization (1). Our efficient DAG-to-DAG conversion (2) then initializes the e-graph. The highlight of our approach lies in the extraction and evaluation phase (3), with our novel simulated annealing algorithm, parallel processing, and dual cost evaluation methods (ABC-based quality-prioritized and GNN-based rapid estimation).

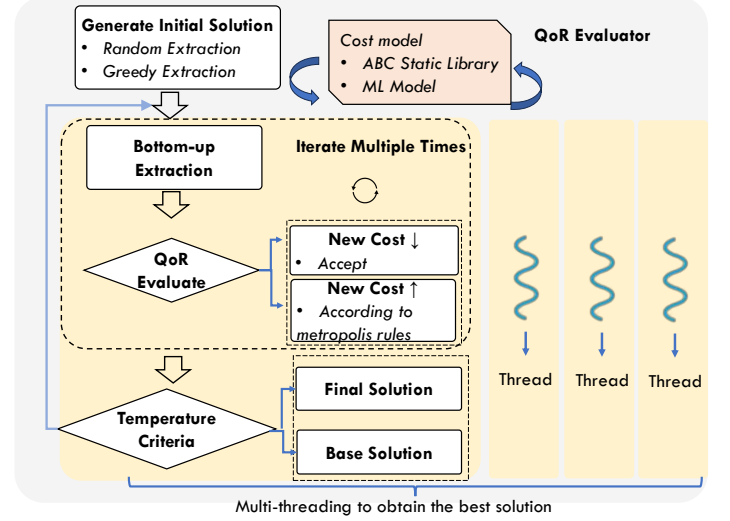


Fig. 4: Simulated-annealing-based extraction algorithm.

B. Efficient Extraction methods

1) Simulated Annealing e-graph Extraction

The extraction phase in e-graph-based optimization is crucial for obtaining high-quality results. However, extraction from equality saturated graphs is known to be NP-hard [18], and traditional greedy algorithms often fail to reflect QoR after circuit technology mapping. To address this challenge, we have developed a novel extraction algorithm based on simulated annealing (SA) [19], a probabilistic technique for approximating the global optimum of a given function. In addition, we integrate Solution Space Pruning and Multi-Threaded Parallel Computing to achieve efficient e-graph extraction and facilitate broad structural exploration.

Our SA extractor combines the principles of bottom-up extraction with simulated annealing to efficiently explore the solution space of an e-graph. This hybrid approach aims to overcome local optima and find high-quality circuit representations. The algorithm (illustrated in Figure 4) operates as follows:

We begin by generating an initial solution using a basic extraction method, such as greedy depth-minimization or random selection. The main loop of the algorithm then iteratively improves this solution while gradually decreasing the temperature parameter of simulated annealing, with specific cooling schedules detailed in Section IV-A. At each iteration, we generate a neighboring solution, evaluate its cost, and decide whether to accept or reject it based on the cost difference and current temperature. This process allows for occasional uphill moves, enabling the algorithm to escape local optima.

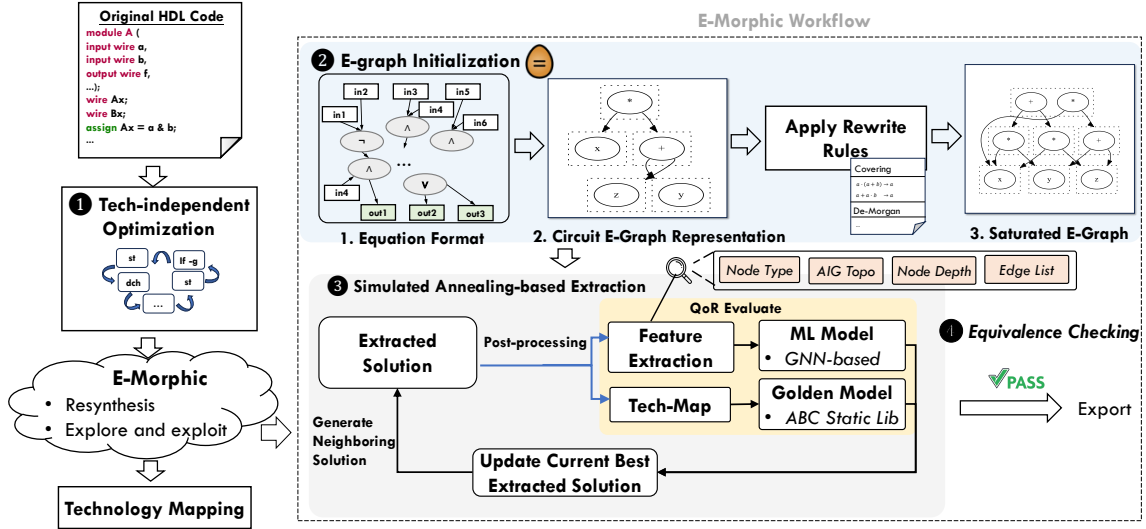


Fig. 5: E-morphic overview.

The neighbor generation process, illustrated in Algorithm 1, starts from leaf nodes and propagates changes upwards through the e-graph. For each node, we decide whether to update its selection based on a combination of cost improvement and random chance. This randomness allows for exploration of different parts of the solution space while ensuring that the solution quality does not degrade below the initial solution.

Algorithm 1: Generate Neighboring Solution

Input: Current solution $\mathcal{O}_{current}$, e-graph $\mathcal{E}$, cost function $\mathcal{C}$, random probability p_{random}

Output: New solution $\mathcal{O}_{new}$

```

1  $\mathcal{O}_{new} \leftarrow \mathcal{O}_{current}$ ;
2  $traversalQueue \leftarrow LeafNodes(\mathcal{E})$ ;
3  $Costs\_map \leftarrow \{\infty, \text{for all e-classes}\}$ ;
4 while  $traversalQueue$  is not empty do
5    $enode \leftarrow traversalQueue.pop()$ ;
6    $class \leftarrow GetClass(enode)$ ;
7    $prev\_cost \leftarrow Costs\_map[class]$ ;
8    $CS \leftarrow$  all children of  $enode$ ;
9   if  $\mathcal{C}$  is a "sum cost" then
10     $new\_cost \leftarrow \mathcal{C}(enode) + \sum_{i \in CS} Costs\_map[i]$ ;
11  end
12  if  $\mathcal{C}$  is a "depth cost" then
13     $new\_cost \leftarrow \mathcal{C}(enode) + \max_{i \in CS} \{Costs\_map[i]\}$ ;
14  end
15  if  $prev\_cost = \infty$  or ( $new\_cost < prev\_cost$  and
     $random() \geq p_{random}$ ) then
16     $\mathcal{O}_{new}.choose(eclass, enode)$ ;
17     $Costs\_map[class] \leftarrow new\_cost$ ;
18     $traversalQueue.extend(GetParents(eclass))$ ;
19  end
20 end
21 return  $\mathcal{O}_{new}$ ;

```

2) Solution Space Pruning for Extraction

Additionally, we employ solution space pruning techniques to accelerate the extraction process. Traditional bottom-up ex-

traction algorithms often traverse every e-node in the e-graph, recalculating costs even when changes are unlikely to yield improvements. In contrast, our method maintains a traversal queue in Algorithm 1 that only records the e-nodes with costs less than or equal to the minimum cost in each e-class, effectively skipping the remaining e-nodes with higher costs, thereby significantly filtering out the numerous redundant e-nodes introduced in each e-class due to commutative and associative laws as shown in Figure 6. In addition, the minimum cost of each e-class is maintained by a hashmap called $Costs_map$, which caches the optimal costs per e-class. This approach avoids redundant computations by preventing the re-evaluation of e-node costs when the costs of their child nodes remain unchanged.

By combining these techniques, we achieve a more efficient and effective extraction process that is particularly well suited for large and complex e-graphs encountered in large circuit optimization scenarios.

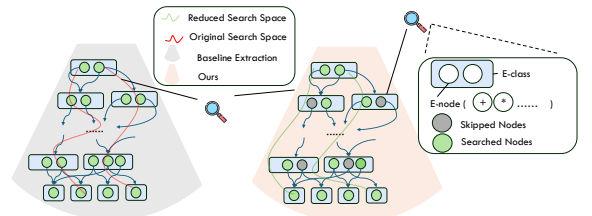


Fig. 6: Solution space pruning.

3) Multithreaded Parallel Cost Evaluation

In the QOR evaluation phase of our simulated annealing algorithm as referenced in Figure 4, we support multi-threading to enhance efficiency. Leveraging initial solutions derived from bottom-up extraction, we concurrently run multiple annealing processes across different threads. This parallel execution allows us to perform QOR fine-tuning on multiple solution sets simultaneously, facilitating extensive resynthesis for a variety of structural transformations. Ultimately, we map all the parallel-generated solutions and select the one with the best QOR results.

C. Dual-Model Approach for Cost Estimation

To balance optimization speed and result quality, we employ two complementary cost models in our extraction process:

- 1) **Runtime-prioritized mode:** This mode primarily uses a Graph Neural Network (GNN) model for rapid estimation. The GNN quickly evaluates circuit characteristics based on node type, topology order, and connection relationships. While less accurate, it provides fast feedback to guide the simulated annealing process, making it particularly useful for exploring many candidate solutions.
- 2) **Quality-prioritized mode:** This mode performs a fast but rough mapping to obtain a QoR estimation. It converts the extracted circuit to an equation file, processes it using ABC with a standard cell library, and applies technology mapping and optimizations. The resulting circuit delay serves as the primary cost metric, ensuring high-quality evaluations at the cost of longer runtime.

By combining these modes, our approach efficiently navigates the complex solution space of e-graphs, balancing speed and accuracy in cost estimation for various circuit optimization tasks.

D. Efficient Implementation

Building upon our novel approach for structural exploration and extraction, we have developed several key techniques to enhance the efficiency of our framework. These improvements address the major bottlenecks in previous e-graph-based logic synthesis methods, allowing E-morphic to handle larger and more complex circuits.

1) Direct DAG-to-DAG Conversion

Previous work in e-graph-based logic synthesis [12] relied on S-expressions as the intermediate representation between circuits and e-graphs. However, S-expressions, being nested lists derived from flattening abstract syntax trees (ASTs), are not ideal for representing circuit graphs. Shared nodes must be duplicated in S-expressions, leading to significant space and time overhead during circuit-to-e-graph conversion and potentially degrading the overall quality of results (QoR). To address this limitation, we have developed a more efficient intermediate format that serves as a bridge for conversion. It eliminates the need for complex parsing and reconstruction steps, significantly reducing computational overhead as illustrated in Figure 8. This format is essentially a serialization of the initial e-graph, using unique identifiers to represent nodes and their relationships. Figure 7 provides an example of this representation. For an equation format input, each assignment corresponds to a node in the initial e-graph, which is referred to by an `id` in our DSL. The one-to-one correspondence between circuit elements and their e-graph counterparts avoids the exponential growth in representation size of S-expressions.

2) Flexible Cost Model Integration

Our framework provides an interface for integrating various cost models, including machine learning-based approaches. We don't limit on the specifics of the particular model, this flexibility allows us to leverage state-of-the-art techniques for rapid quality assessment. Later, Section IV-D will demonstrate the use of

```
"egraph" : {
  "3": {"id": 3, "nodes": [{"Symbol": "a"}], "parents": [7,8]},
  ... // node id:(b, id:4), (c, id: 5)
  "7": {"id": 7, "nodes": [{"AND": [3,4]}], "parents": [6,9]},
  "8": {"id": 8, "nodes": [{"AND": [3,5]}], "parents": [9]},
  ... } // other nodes
```

Fig. 7: The intermediate format for efficient e-graph—circuit conversion in pre- and post-processing.

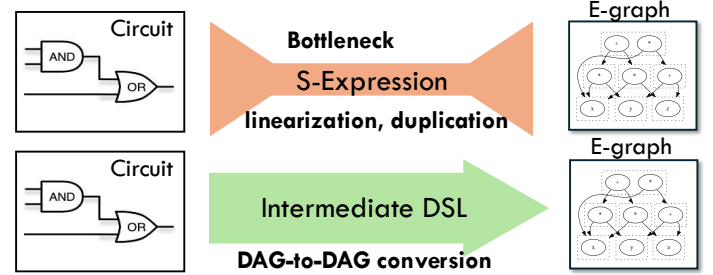


Fig. 8: Prior and our new e-graph—circuit conversion.

a GNN model to provide rapid cost estimation to guide the simulated annealing process.

By combining these efficiency-enhancing techniques, E-morphic achieves significant speed-ups, enabling the application of e-graph-based optimization to a wider range of circuit sizes and complexities.

IV. EXPERIMENTS

A. Experiment Setting

We implement the E-morphic framework in Rust to interface with the existing `egg` library for e-graphs. All experiments are performed on a Ubuntu 20.04.4 LTS server with dual Intel Xeon Platinum 8375C processors and 256GB memory. The test circuits all come from the EPFL benchmark suite [20], containing 2 small-, 2 medium- and 6 large-scale cases. Throughout these experiments, we use ASAP 7nm [21] technology library to evaluate the post-mapping QoR. E-graph rewriting takes 5 iterations, which already produces a substantial number of equivalence classes to facilitate structural exploration. In the quality-prioritized mode, we utilize 4 threads, with the annealing exit condition set to 4 iterations. In the first iteration, the temperature T is set to 2000 as a high temperature T_1 encourages acceptance of inferior solutions, helping to avoid local optima. For the 2nd and 3rd iterations, the temperature is defined as $T_n = T_{n-1} \times \frac{[\text{new cost} - \text{old cost}]}{n \times 10000}$ (n is the number of iterations), ensuring progression into the pseudogreedy local-search stage. In the 4th iteration, the temperature is adjusted to $T_n = T_{n-1} \times \frac{[\text{new cost} - \text{old cost}]}{n}$, allowing for further enhancements in solution quality. For the runtime-prioritized mode, we utilize 6 threads to balance the performance loss of the cost model. All the circuits optimized through E-morphic are verified for equivalence using the `cec` command in ABC.

B. Efficient DAG-to-DAG Conversion

One bottleneck of the prior work E-Syn is the conversion between circuit and e-graphs. Here we compare the circuit-e-graph mutual conversion time using the benchmark circuits. As can be seen from Table III, in most cases, our direct DAG-to-DAG conversion takes only a fraction of a second. Whereas,

TABLE II: QoR and runtime comparison between E-morphic and the baseline.

Circuit	SOP Balancing Baseline				SOP Balancing + E-morphic w/o ML model				SOP Balancing + E-morphic w ML model			
	Area (μm^2)	Delay (ps)	lev	runtime (s)	Area (μm^2)	Delay (ps)	lev	runtime (s)	Area (μm^2)	Delay (ps)	lev	runtime (s)
hyp	370305.16	253779.06	10929	1496.23	386483.84	240435.12	10948	2102.52	366751.38	248781.94	11033	1646.86
div	56873.2	25378.59	1258	108.04	34475.98	24747.77	1300	280.79	48386.24	24333.65	1298	147.77
mem_ctrl	29433.64	1281.99	61	76.5	29219.49	1056.09	59	113.17	29365.29	1081.48	60	97.09
log2	41154.32	5233.86	231	161.49	32840.69	5033.46	240	289.79	41234.11	5153.74	233	208.30
multiplier	35799.38	2632.08	147	54.68	31443.34	2371.24	130	105.02	35002.03	2599.37	147	87.15
sqrt	53262.02	78627.99	3574	112.27	33852.66	72796.83	3578	270.21	48909.25	76432.02	3579	143.43
square	21240.14	834.01	69	25.12	21418.84	792.09	69	50.22	21532.44	794.16	70	37.46
arbiter	4464.75	288.18	23	12.58	4587.45	270.98	22	23.34	4532.63	284.27	24	20.18
sin	16052.7	3712.41	119	34.95	15737.77	3567.12	113	62.31	15737.77	3685.34	118	50.83
adder	1206.99	584.53	57	1.96	1282.11	536.42	57	10.2	1285.61	546.86	57	7.07
GEOMEAN	25274.02	5620.01	292	60.32	22104.32	5210.55	287	126.25	24660.84	5390.13	295	91.42
Improvements					12.54%	7.29%			2.42%	4.09%		

TABLE III: Comparison of e-graph-circuit conversion with a time-limit of 3600 seconds and 8 GB memory-limit. “Forward” means circuit to e-graph conversion and “backward” refers to e-graph to circuit conversion. **TO** represents “timeout.” **MO** stands for “out-of-memory.”

Design	#. e-node	E-Syn [12]		E-morphic (this work)	
		Forward (s)	Backward (s)	Forward (s)	Backward (s)
hyp	420897	TO & MO	N.A.*	8.2	4.6
div	101860	TO & MO	N.A.*	1.57	1.2
mem_ctrl	84701	TO & MO	N.A.*	1.44	0.8
log2	54532	TO & MO	N.A.*	0.93	0.71
multiplier	50761	TO	N.A.*	0.84	0.75
sqrt	41234	TO	N.A.*	0.70	0.53
square	35685	TO	N.A.*	0.59	0.55
arbiter	23619	59.53	65.56	0.38	0.12
sin	8948	TO	N.A.*	0.15	0.13
adder	2548	4.25	53.4	0.04	0.04
GEOMEAN	-	-	-	0.65	0.46

* Not available due to the prior e-graph to circuit conversion failure.

E-Syn hardly scales to over 10^4 e-nodes, resulting in time-out in most cases. The results indicate that our conversion method is minimally affected by the circuit size. Even when applied to circuits over 10^5 e-nodes, the conversion can still be performed in a remarkably short runtime.

C. Enhancing Conventional Delay-oriented Synthesis

As delay-oriented optimization is crucial to meet timing constraints in logic synthesis, in this experiment, we compare our approach against a competitive delay-oriented logic optimization flow published in [22], which has been adopted by industrial users. The optimization sequence is as follows: *(st; if -g -K 6 -C 8)(st; dch; map)*⁴. It takes advantage of SOP balancing [22], priority-cut-based mapping [23] and resynthesis after mapping [1]. We argue that it is nontrivial to outperform this flow with further optimization. For comparison, the E-morphic flow applies the same *(st; if -g -K 6 -C 8)(st; dch; map)*³ first. Before the final round of *(st; dch; map)*, we make use of e-graph rewriting to obtain a set of structurally diverse solutions. The results are shown in Table II. Compared to the conventional delay-oriented optimization flow, E-morphic achieves delay reduction on all designs, averaging 7.29%, along with an area saving of nearly 12.54% while the runtime overhead is moderate. Due to space limit, we omit the result of E-Syn [12] as it already times out for all large-scale circuits in this benchmark.

D. GNN-based Cost Model for Faster Extraction

To further reduce the runtime overhead of E-morphic, one can plug in a machine learning model for rapid cost evaluation. Here we demonstrate the use of state-of-the-art HOGA [24]

model. We firstly train it on circuits from the OpenABC-D benchmark [25] synthesized by the aforementioned flow to adapt its prediction towards the setting in this experiment. Each design module in the OpenABC-D benchmark generates 100 different structural samples, resulting in a training set of around 40,000 samples. The labels are obtained using the same ASAP 7nm cell library for technology mapping. The delay prediction achieves a Mean Absolute Percentage Error (MAPE) of 25.2% and a Kendall’s τ of 0.62. Using this prediction model, we present logic synthesis results in the last column of Table II, which shows an averaged runtime saving of nearly 28%.

E. Runtime Analysis

We plot the proportion of time consumption of each step in Figure 9. It can be seen that the time spent on the existing technology-independent optimization and mapping in the delay-oriented flow [22] accounts for a large portion of the total runtime, which is irrelevant to the usage of e-graph optimization. While the time for the additional operations in E-morphic (mainly conversion and simulated annealing) is moderate, demonstrating the efficiency of our framework. Particularly, this efficiency is more pronounced in large-scale circuits, where the proportion of our time overhead is significantly lower.

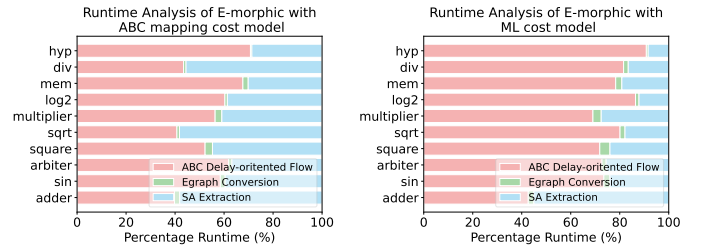


Fig. 9: Runtime breakdown of E-morphic logic synthesis

V. CONCLUSION

This paper proposes a scalable framework named E-morphic, which uses equality saturation for structure exploration before technology mapping. It features efficient e-graph-circuit conversion, solution-space pruning and simulated annealing for extraction. Additionally, it allows the integration of ML prediction models to further reduce the runtime. The techniques presented in this paper will not only benefit applications of equality saturation in logic synthesis, but also other e-graph-based large-scale optimization problems.

REFERENCES

- [1] S. Chatterjee, A. Mishchenko, R. Brayton, X. Wang, and T. Kam. Reducing structural bias in technology mapping. In *ICCAD-2005. IEEE/ACM International Conference on Computer-Aided Design, 2005.*, pages 519–526, 2005.
- [2] Eric Lehman, Yosinori Watanabe, Joel Grodstein, and Heather Harkness. Logic decomposition during technology mapping. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 16(8):813–834, 1997.
- [3] Alan Mishchenko, Satrajit Chatterjee, and Robert Brayton. Improvements to technology mapping for lut-based fpgas. In *Proceedings of the 2006 ACM/SIGDA 14th international symposium on Field programmable gate arrays*, pages 41–49, 2006.
- [4] Leon Stok, Andrew J. Sullivan, and Mahesh A. Iyer. Wavefront technology mapping. In *1999 Design, Automation and Test in Europe (DATE '99), 9-12 March 1999, Munich, Germany*, page 531. IEEE Computer Society / ACM, 1999.
- [5] Eric Lehman, Yosinori Watanabe, Joel Grodstein, and Heather Harkness. Logic decomposition during technology mapping. *IEEE Trans. Comput. Aided Des. Integr. Circuits Syst.*, 16(8):813–834, 1997.
- [6] Ecenur Ustun, Cunxi Yu, and Zhiru Zhang. Equality saturation for datapath synthesis: A pathway to pareto optimality. In *2023 60th ACM/IEEE Design Automation Conference (DAC)*, pages 1–2. IEEE, 2023.
- [7] Ross Tate, Michael Stepp, Zachary Tatlock, and Sorin Lerner. Equality saturation: A new approach to optimization. In *POPL*, page 264–276, New York, NY, USA, jan 2009. ACM.
- [8] Jianyi Cheng, Samuel Coward, Lorenzo Chelini, Rafael Barbalho, and Theo Drane. SEER: Super-optimization explorer for HLS using e-graph rewriting with MLIR. *ASPLOS*, 2024.
- [9] Yan Pi, Hongji Zou, Tun Li, Wanxia Qu, and Hai Wan. ESFO: Equality saturation for FIRRTL optimization. In *GLSVLSI*, 2023.
- [10] Samuel Coward, George A Constantinides, and Theo Drane. Automating constraint-aware datapath optimization using e-graphs. In *DAC*, 2023.
- [11] Andy Wanna, Samuel Coward, Theo Drane, George A. Constantinides, and Milos D. Ercegovac. Multiplier optimization via e-graph rewriting. *ACSSC*, 2023.
- [12] Chen Chen, Guangyu Hu, Dongsheng Zuo, Cunxi Yu, Yuzhe Ma, and Hongce Zhang. E-Syn: E-graph rewriting with technology-aware cost functions for logic synthesis. In *DAC*, 2024.
- [13] Alan Mishchenko, Satrajit Chatterjee, and Robert K. Brayton. Dag-aware AIG rewriting a fresh look at combinational logic synthesis. In Ellen Sentovich, editor, *Proceedings of the 43rd Design Automation Conference, DAC 2006, San Francisco, CA, USA, July 24-28, 2006*, pages 532–535. ACM, 2006.
- [14] Aaron R Bradley and Zohar Manna. *The calculus of computation: decision procedures with applications to verification*. Springer Science & Business Media, 2007.
- [15] Ecenur Ustun, Ismail San, Jiaqi Yin, Cunxi Yu, and Zhiru Zhang. Impress: Large integer multiplication expression rewriting for FPGA HLS. In *FCCM*, pages 1–10. IEEE, 2022.
- [16] Max Willsey, Chandrakana Nandi, Yisu Remy Wang, Oliver Flatt, Zachary Tatlock, and Pavel Panchekha. egg: Fast and extensible equality saturation. *POPL*, 2021.
- [17] Samuel Coward, George A Constantinides, and Theo Drane. Automatic datapath optimization using e-graphs. In *ARITH*, pages 43–50. IEEE, 2022.
- [18] Franz Baader and Tobias Nipkow. *Term Rewriting and All That*. Cambridge University Press, 1998.
- [19] Scott Kirkpatrick, C Daniel Gelatt Jr, and Mario P Vecchi. Optimization by simulated annealing. *science*, 220(4598):671–680, 1983.
- [20] Luca Amarú, Pierre-Emmanuel Gaillardon, and Giovanni De Micheli. The EPFL combinational benchmark suite. In *IWLS*, number CONF, 2015.
- [21] Lawrence T Clark, Vinay Vashishtha, Lucian Shifren, Aditya Gujja, Saurabh Sinha, Brian Cline, Chandarasekaran Ramamurthy, and Greg Yeric. ASAP7: A 7-nm FinFET predictive process design kit. *Microelectronics Journal*, 53:105–115, 2016.
- [22] Alan Mishchenko, Robert Brayton, Stephen Jang, and Victor Kravets. Delay optimization using sop balancing. In *Proceedings of the International Conference on Computer-Aided Design, ICCAD '11*, page 375–382. IEEE Press, 2011.
- [23] Alan Mishchenko, Sungmin Cho, Satrajit Chatterjee, and Robert Brayton. Combinational and sequential mapping with priority cuts. In *ICCAD*, pages 354–361. IEEE, 2007.
- [24] Chenhui Deng, Zichao Yue, Cunxi Yu, Gokce Sarar, Ryan Carey, Rajeev Jain, and Zhiru Zhang. Less is more: Hop-wise graph attention for scalable and generalizable learning on circuits. In *DAC*, 2024.
- [25] Animesh Basak Chowdhury, Benjamin Tan, Ramesh Karri, and Siddharth Garg. Openabc-d: A large-scale dataset for machine learning guided integrated circuit synthesis, 2021.